

Lab 8 – Quy hoạch động & knapsack

1. Mục tiêu

Kết thúc Lab 8, sinh viên có thể:

- Nhớ và cài đặt lại knapsack 0/1 bằng DP bảng 2 chiều (bottom-up).
- Viết hàm truy vết để lấy ra danh sách item được chọn từ bảng 2D.
- Tối ưu bộ nhớ từ DP 2D sang DP 1D, hiểu vì sao phải duyệt w ngược.
- Nhận thức trade-off giữa “chỉ cần giá trị tối ưu” và “cần cả nghiệm tối ưu” khi tối ưu bộ nhớ.

2. Cấu trúc Lab

Bài 1 – Knapsack 0/1 với DP 2D + truy vết

File: lab8_bai1_knapsack_2d.py

1.1. Bài toán

Cho:

- Danh sách values: giá trị từng món.
- Danh sách weights: khối lượng từng món.
- Sức chứa balo W.

Yêu cầu:

- Tính giá trị tối đa có thể mang.
- Xác định **danh sách item** được chọn (index).

Dùng DP 2D như đã học trên lớp.

1.2. Hàm xây bảng dp 2D

```
def knapsack_dp_table(values, weights, W):  
    """  
    Xây bảng dp 2D cho knapsack 0/1.  
    dp[i][w]: max value dùng i món đầu, sức chứa w.  
    """  
    n = len(values)  
    dp = [[0] * (W + 1) for _ in range(n + 1)]  
  
    for i in range(1, n + 1):  
        wt = weights[i - 1]  
        val = values[i - 1]  
        for w in range(1, W + 1):  
            if wt > w:  
                dp[i][w] = dp[i - 1][w]  
            else:  
                dp[i][w] = max(  
                    dp[i - 1][w],  
                    val + dp[i - 1][w - wt]  
                )  
    return dp
```

1.3. Hàm lấy max value từ bảng

```
def knapsack_max_value_2d(values, weights, W):  
    dp = knapsack_dp_table(values, weights, W)  
    n = len(values)  
    return dp[n][W]
```

1.4. Hàm truy vết danh sách item từ dp 2D

Dựa trên bảng dp:

- Bắt đầu từ $i = n$, $w = W$.
- Nếu $dp[i][w] == dp[i-1][w] \rightarrow$ item i không được chọn.
- Nếu khác $\rightarrow$ item i được chọn, trừ w .

```
def knapsack_trace_items_2d(values, weights, W):  
    dp = knapsack_dp_table(values, weights, W)  
    n = len(values)  
    w = W  
    chosen = []  
  
    i = n  
    while i > 0 and w > 0:  
        if dp[i][w] == dp[i - 1][w]:  
            # không lấy item i-1  
            i -= 1  
        else:  
            # lấy item i-1  
            chosen.append(i - 1)  
            w -= weights[i - 1]  
            i -= 1  
  
    chosen.reverse()  
    max_val = dp[n][W]  
    return max_val, chosen
```

1.5. Hàm test cho bài 1

```
def test_knapsack_2d():
    values = [1, 4, 5]
    weights = [1, 2, 3]
    W = 5

    dp = knapsack_dp_table(values, weights, W)
    print("Bảng dp (hàng: i, cột: w):")
    for row in dp:
        print(row)

    max_val, items = knapsack_trace_items_2d(values, weights, W)
    print("\nMax value:", max_val)
    print("Các item chọn (index):", items)
    print("Chi tiết:")
    total_weight = sum(weights[i] for i in items)
    print(" Tổng weight:", total_weight)
    for i in items:
        print(f" item {i}: wt={weights[i]}, val={values[i]}")

if __name__ == "__main__":
    test_knapsack_2d()
```

Bài 2 – Knapsack 0/1 với DP 1D

File: lab8_bai2_knapsack_1d.py

2.1. Mục tiêu

- Viết lại knapsack 0/1 dùng **mảng 1 chiều**: `dp[w]`.
- Hiểu vì sao phải duyệt w từ $W \rightarrow 0$.

2.2. Hàm knapsack 1D

```
def knapsack_1d(values, weights, W):  
    """  
    Knapsack 0/1 dùng dp 1 chiều.  
    dp[w]: max value với sức chứa w.  
    """  
    n = len(values)  
    dp = [0] * (W + 1)  
  
    for i in range(n):  
        wt = weights[i]  
        val = values[i]  
        # Duyệt ngược để không dùng 1 món nhiều lần  
        for w in range(W, wt - 1, -1):  
            dp[w] = max(dp[w], val + dp[w - wt])  
  
    return dp[W]
```

- Nếu duyệt w từ 0 $\rightarrow$ W, `dp[w - wt]` có thể đã được cập nhật với item i $\rightarrow$ dùng item i nhiều lần $\rightarrow$ sai với 0/1.

2.3. Hàm test so sánh 2D vs 1D

```
from lab8_bai1_knapsack_2d import knapsack_max_value_2d  
  
def test_knapsack_1d():  
    values = [1, 4, 5]  
    weights = [1, 2, 3]  
    W = 5  
  
    ans_2d = knapsack_max_value_2d(values, weights, W)
```

```

ans_1d = knapsack_1d(values, weights, W)

print("Max value 2D:", ans_2d)
print("Max value 1D:", ans_1d)

if __name__ == "__main__":
    test_knapsack_1d()

```

Yêu cầu:

- Kết quả 2D và 1D phải giống nhau trên nhiều test khác nhau.

Bài 3 – Knapsack 1D + truy vết nghiệm

File: lab8_bai3_knapsack_1d_trace.py

3.1. Mục tiêu

- Kết hợp lợi ích dp 1D (tối ưu value) với truy vết nghiệm.
- Cách để thực hiện trong lab:
 - Dùng dp 1D cho value.
 - Dùng thêm bảng `choice[i][w]` để ghi nhận “item i có được chọn tại capacity w hay không”.

3.2. Hàm knapsack 1D với bảng choice

```

def knapsack_1d_with_choice(values, weights, W):
    """
    DP 1D + bảng choice để truy vết.
    dp[w]: max value với sức chứa w.
    choice[i][w] = True nếu tại item i, capacity w, ta chọn item i.
    """
    n = len(values)

```

```

dp = [0] * (W + 1)
choice = [[False] * (W + 1) for _ in range(n)]

for i in range(n):
    wt = weights[i]
    val = values[i]
    # Duyệt ngược w
    for w in range(W, wt - 1, -1):
        # Nếu lấy item i
        take_val = val + dp[w - wt]
        if take_val > dp[w]:
            dp[w] = take_val
            choice[i][w] = True
        # Nếu không lấy → dp[w] giữ nguyên, choice[i][w] = False (default)

return dp, choice

```

3.3. Hàm truy vết items từ dp + choice

```

def reconstruct_items_1d(values, weights, W):
    dp, choice = knapsack_1d_with_choice(values, weights, W)
    n = len(values)
    w = W
    chosen = []

    i = n - 1
    while i >= 0 and w > 0:
        if choice[i][w]:
            chosen.append(i)
            w -= weights[i]

```

```
i -= 1

chosen.reverse()
max_val = dp[W]
return max_val, chosen
```

3.4. Hàm test truy vết nghiệm 1D

```
def test_knapsack_1d_trace():
    values = [1, 4, 5]
    weights = [1, 2, 3]
    W = 5

    max_val, items = reconstruct_items_1d(values, weights, W)

    print("Max value:", max_val)
    print("Các item chọn (index):", items)
    total_weight = sum(weights[i] for i in items)
    print("Tổng weight:", total_weight)
    for i in items:
        print(f" item {i}: wt={weights[i]}, val={values[i]}")

if __name__ == "__main__":
    test_knapsack_1d_trace()
```

Sinh viên:

- So sánh tập item từ 1D + choice với tập item từ 2D (bài 1).
- Đảm bảo kết quả giống nhau.